Imperial College London

RCS RESEARCH COMPUTING SERVICE
Gateway to Discovery

Demystifying HPC for beginners

Katerina Michalickova – kmichali@imperial.ac.uk

Early Career Researcher Institute, Research Computing and Data Science

IMPERIAL
Early Career Researcher Institute

Outline for today

- introduction, key links and contacts
- preparing to use the HPC cluster
  - working remotely and login
  - software management on the cluster
  - copying files to and from the cluster
- software execution on the cluster
- interactive resources

Imperial College London

Why are we here today?

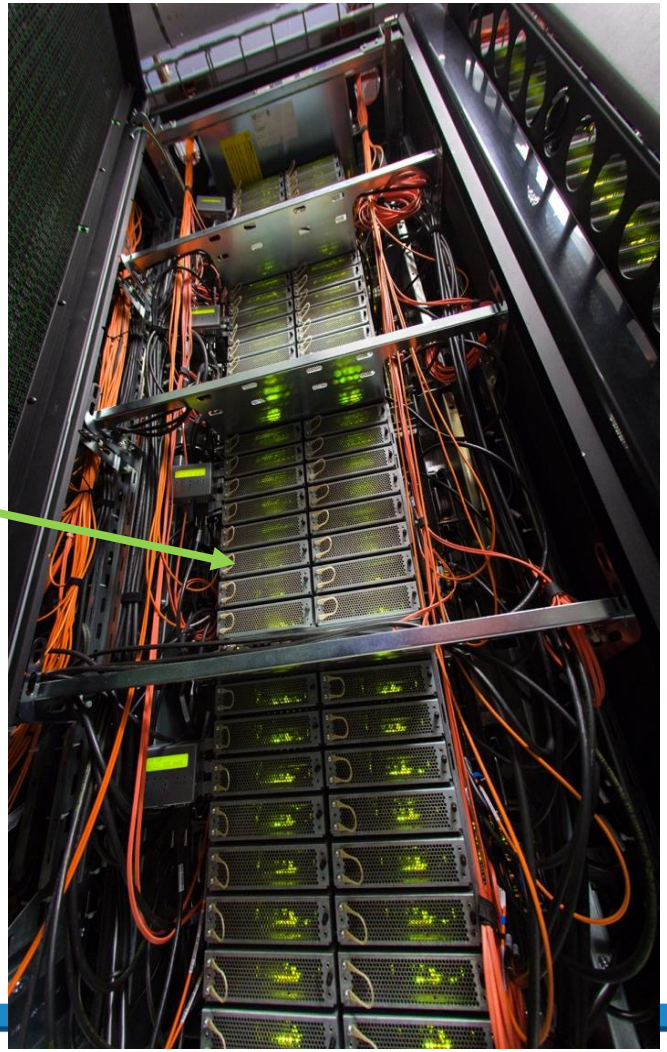
- Pillars (legs) of science today – theory, experimentation *and computing*.
- Much of scientific experimentation such as modeling, simulations or data management and mining is made possible by computing.
- If you need to use a central resource, it is very likely it will come in a form of **high performance computing cluster**.

Computer cluster

- A **computer cluster** consists of a set of connected computers that work together so that, in many respects, they can be viewed as a single system.

Inside a compute rack

a single computer
aka a compute node



Inside a disk rack

All compute nodes see one central file system – the Research Data Store

Imperial College London

What can clusters do?

- the cluster can serve to offload code execution from your laptop/workstation
  - code that runs too long or needs too much memory or disk space
- clusters are particularly useful for executing parallel code
  - on one compute node
  - on multiple compute nodes at once

- Note on speed of execution:
- the compute nodes have similar architecture to your desktop - they are not much faster
- the main advantage of cluster computing lies in parallel code execution

HPC terminology

I will try to avoid or explain again during the class. In case I forget myself, here are few terms that need explanation:

- **Node** = compute node = group of cores that can access the same memory (I may inadvertently also say a computer or a machine)
- **Memory** = main memory or RAM - fast memory directly connected to the processor, when your program is running it is stored in RAM together with needed data
- **Core** = the basic computation unit inside a processor that can run a single process
- **Serial code** = runs on one core
- **Parallel code** = program that runs on two or more cores
- **Job** = your program on the cluster
- **Submit job** = instruct the cluster to run your program

Workload types

The resource contains hardware components suited to different type of computations:

- High-throughput tasks – large number of relatively small jobs
- Parallel jobs on a single node
- Parallel jobs on multiple nodes
- Large single node memory jobs (up to TBs of memory)
- Jobs using GPUs (such as machine learning tasks)

Key contacts

- RCS website - https://www.imperial.ac.uk/admin-services/ict/self-service/research-support/rcs/
- User support - https://www.imperial.ac.uk/admin-services/ict/self-service/research-support/rcs/get-support/contact-us/
- Technical documentation https://icl-rcs-user-guide.readthedocs.io/en/latest/
- Weekly drop-in clinics - https://icl-rcs-user-guide.readthedocs.io/en/latest/support/attend-a-clinic/
- Service status (under Zscaler)- https://selfservice.rcs.imperial.ac.uk/service-status
- Self-service (under Zscaler, interactive access, group management, job monitoring and extensions etc.) - selfservice.rcs.imperial.ac.uk

Outline for today

- introduction, key links and contacts
- preparing to use the HPC cluster
  - working remotely and login
  - software management on the cluster
  - copying files to and from the cluster
- software execution on the cluster
- interactive resources

Imperial College London

Before you login

- use **Zscaler** when off-campus
https://www.imperial.ac.uk/admin-services/ict/self-service/connect-communicate/remote-access/unified-access/

- on campus - use **Imperial-WPA wifi**

- for **regular HPC access**, register as follows
https://www.imperial.ac.uk/admin-services/ict/self-service/research-support/rcs/get-access/

- some of you have only temporary access for this course - it expires on the first of each month

- you'll need a terminal application
  - Terminal on Mac
  - OpenSSH, Putty or Windows Linux Subsystem on Windows)

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/getting-started/using-ssh/

Log in using your terminal

In a terminal application, type the following:

```
ssh your_username@login.cx3.hpc.ic.ac.uk
```

- This course uses the cx3 system (aka cx3_phase2)
- The documentation:

https://icl-rcs-user-guide.readthedocs.io/en/latest/

Imperial College London

RCS RESEARCH COMPUTING SERVICE
Gateway to Discovery

HPC environment

- Linux operating system
- Bash shell

kmichali — kmichali@login-ai:~ — ssh kmichali@login.cx3.hpc.ic.ac.uk — 103×44

```
dyn22109-48:~ kmichali$ ssh kmichali@login.cx3.hpc.ic.ac.uk
kmichali@login.cx3.hpc.ic.ac.uk's password:
                          Imperial College London Research Computing Service
                          --------------------------------------------------
  ##############################################################################################
  #                                                                                            #
  #       Continue ONLY if you are an authorised user, otherwise EXIT now                      #
  #                                                                                            #
  #       Communications, including personal communications, made on or through  Imperial      #
  #       College's computing and telecommunications systems may be monitored or recorded      #
  #       to  secure effective  system operation  and for other lawful purposes.  See the      #
  #       Information Systems Security Codes of Practice for more information.                 #
  #                                                                                            #
  #                       Remember the logins are SHARED RESOURCES.                            #
  #                           Programs that are long-running,                                  #
  #                           I/O intensive,                                                   #
  #                          memory-intensive or multithreaded                                 #
  #                              MUST BE RUN AS BATCH JOBS.                                    #
  #                                                                                            #
  ##############################################################################################

  * Service Status:       https://selfservice.rcs.imperial.ac.uk/service-status
  * CX3 Documentation:    https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/pilot/cx3-phase2/
  * RCS Support:          https://icl-rcs-user-guide.readthedocs.io/en/latest/support/

-------------------------------------------------------------------------------------------------

Last login: Thu Oct 30 14:14:56 2025 from 2a0c:5bc0:81:1e01:25d7:c1e6:2469:1b16
[kmichali@login-ai ~]$
```

Demystifying HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
  - working remotely and login
  - software management on the cluster
  - copying files to and from the cluster
- software execution on the cluster
- interactive resources

Module system for centrally installed packages

- Lmod system sets required environment to use a software package

- List all modules available
  - ml av or ml ov (available or overview)
- Find a module for your software
  - ml av matlab or ml spider matlab
- Load a module
  - ml MATLAB
- Load a specific version
  - ml MATLAB/2024b
- List all loaded modules
  - ml
- Swap modules
  - ml switch MATLAB/2024b MATLAB/2023b
- Unload a module
  - ml unload matlab/R2018a
- Get rid of all loaded modules
  - ml purge

```
anaconda3/2.4.1
anaconda3/4.1.1
anaconda3/4.3.1(default)
anaconda3/personal
angsd/609
angsd/915
annovar/2013-03-18
annovar/2013-08-05
annovar/2015-06-17(default)
ansys/16.2
ansys/17.1
ansys/18.1
ansys/18.2
ansys/19.1-fluids
ansys/cfx/14.0(default)
ansys/cfx/15.0
ansys/cfx/16.1
ansys/fluent/14.0(default)
ansys/fluent/15.0
ansys/workbench/16.1
ant/1.6.1
ant/1.6.5
ant/1.8.1(default)
ants/2.2.0
ANTs/2015-02-23
aracne/2
armadillo/1.1.90
armadillo/1.2.0
armadillo/2.0.1
armadillo/3.2.4
armadillo/3.4.2
armadillo/6.200.2
armadillo/7.200.2(default)
arpack/96-patch(default)
aspect/1.4.0
aspect/1.5.0(default)
aster/11.3
atat/2.71(default)
ATK/2015.1
magma/2.18-5
magma/2011-07-05
magma/devel-modules(default)
make/3.82(default)
maker/2.10
maker/2.31.9
mamba/1.0.0
mantra/2012-11-19
maple/2016
mapsembler/2.2.4
mapsplice/2.2.0
maq/0.7.1(default)
marv/1.0.4
masurca/2.3.2
masurca/3.2.1
masurca/3.2.1_01202017
matlab/R2017a(default)
matlab/R2017b
matlab/R2018a
matplotlib/0.90.1(default)
mauve/2015-02-13
maxima/5.40.0(default)
mayavi/1.5(default)
mcarts/1.2.0
mcl/14.137
mcr/717(default)
mcscan/0.8
mcstas/2.1(default)
meep/0.10
meep/0.10.1
meep/0.20.3
meep/0.20.4
meep/1.0.3
meep/1.1.1(default)
meep/1.3
meep/1.4.3
meme/4.3.0
meme/4.3.0-nompi
meme/4.9.0
```

Cannot find what you're looking for?

- submit a request for installation (variable waiting times): https://www.imperial.ac.uk/admin-services/ict/self-service/research-support/rcs/get-support/contact-us/

- you can install packages in your home directory (take advantage of various pre-installed libraries via module system)

Python and R packages management

- You can manage R and Python either with Anaconda or Easybuild:

- **Anaconda package manager**
  - Simpler to use, possible not as performant, and prone to version resolution issues
  - You install all software yourself
- **Centrally installed software through Easybuild**
  - Always optimised, slightly more complicated setup
  - Most is installed for you, sone you do yourself (via pip or inside R)

Anaconda for Python and R

To begin using Anaconda, you need to install miniforge (minimal version of anaconda with no license restrictions). Install with:

```
module load miniforge/3
miniforge-setup
```

After the installation, you should see **`miniforge3`** directory in your home directory.

There is no module command for using miniforge, any subsequent loading is done with:

```
eval "$(~/miniforge3/bin/conda shell.bash hook)”
```

Using python with Anaconda

Make sure that you have miniforge installed, see previous slide.

Load as follows:

```
eval "$(~/miniforge3/bin/conda shell.bash hook)"
```

Anaconda enables you to create separate package environments for your projects. This helps to avoid version and dependency conflicts.

```
conda create -n projectx          #create new environment
conda activate projectx           #activate env. (conda/source)
conda install scipy               #install scipy
conda deactivate                  #deactivate current env.
```

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/applications/guides/python/

Setting up using installed python modules

```
ml tools/prod #always has to be loaded
ml spider scipy-bundle #examine available scientific python bundle versions
#load scientific bundle, always pick up the latest
ml SciPy-bundle/2025.07-gfbf-2025.07
pip3 list #check list of packages

#proceed to install your own python dependencies in a separate virtual envs
mkdir -p  ~/venv #if it does not exist, make a directory for virtual envs
virtualenv ~/venv/example_env #make a virtual env "example_env"
source ~/venv/example_env/bin/activate #activate example_env
python -m pip install some_python_module #use pip to install what you need
deactivate
```

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/applications/guides/python/

Python modules are downloaded from Easybuild - a community-based repository of optimised software, libraries and packages for HPC - https://easybuild.io/

Using R with Anaconda

R and libraries can be installed using personal Anaconda.

Setup a new conda environment and install R inside it. Packages can be installed using conda install or from within R (plus other libs, e.g. bioconductor-teqc)

```
eval "$(~/miniforge3/bin/conda shell.bash hook)"
conda create -n Renv
conda activate Renv                # (conda or source)
conda install r -c r-png           #install R in new env.

conda install bioconductor-teqc
conda deactivate
```

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/applications/guides/R/

Setting up R with Easybuild

```
ml tools/prod #always have to be loaded
ml spider R-bundle-ICL
ml R-bundle-ICL/2025.10-foss-2024a #load ICL R-bundle that contains multitude of libraries; use "installed.packages() in R to check"
```

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/applications/guides/R/

Application-specific guidance

Information for selected popular applications:

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/applications/guides/

Demystifying HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
  - working remotely and login
  - software management on the cluster
  - copying files to and from the cluster
- software execution on the cluster
- interactive resources

Imperial College London

Before you start

- If not on campus, use Imperial **Zscaler**
- On campus - use **Imperial-WPA wi-fi**

File management

All computers in the HPC resource are connected to one parallel filesystem – Research Data Store (RDS). You get access to:

your home directory ($HOME)	• personal working space • 1 TB allocation (up to 10 million files)
temporary storage ($EPHEMERAL)	• additional individual working space • 10TB • files deleted after 30 days
allocated project space ($RDS_PROJECT)	• your project allocations (if your supervisor has any)

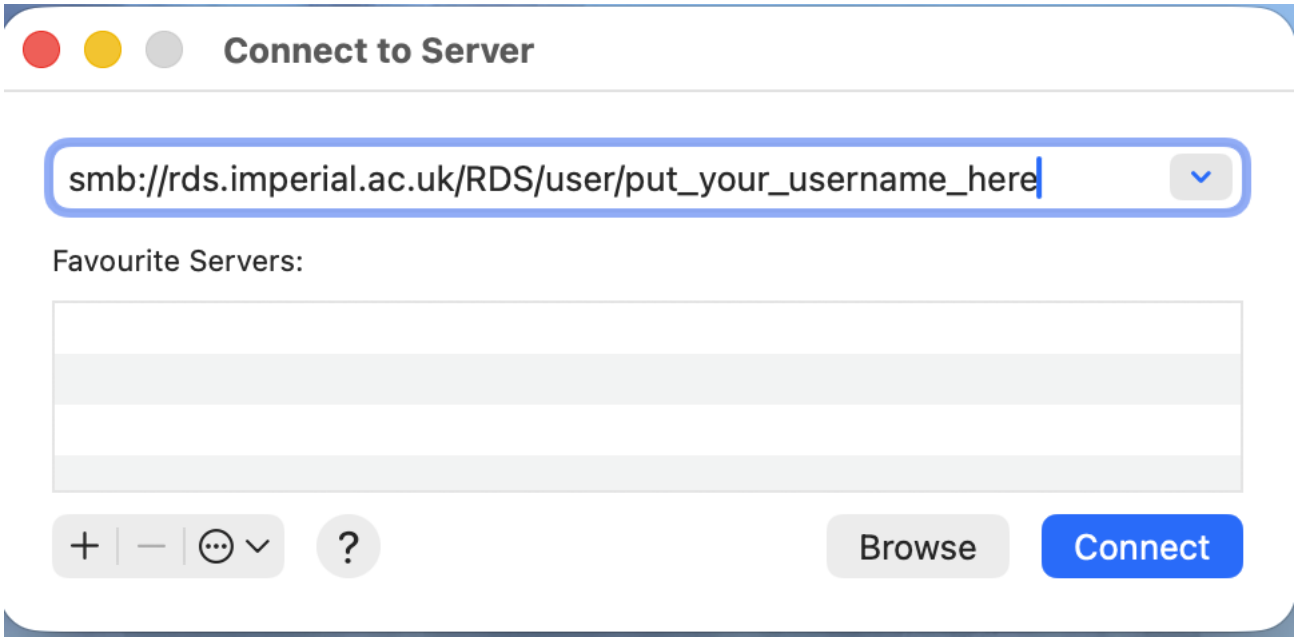
check your usage with “quota -s”

documentation: https://icl-rcs-user-guide.readthedocs.io/en/latest/rds/

1a. Connect to RDS with Finder on your Mac

- **on a Mac (recent macOS version)**
- Finder -> Go -> Connect to Server -> Server Address

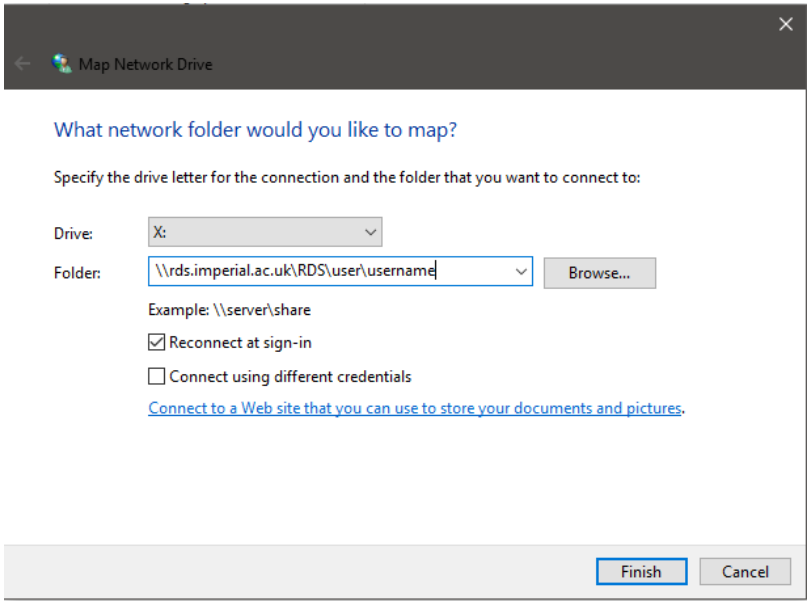
`smb://rds.imperial.ac.uk/RDS/user/your_username_goes_here`



1b. Connect to RDS with File Explorer on Windows

- **recent update of Windows 10**
- File Explorer -> My PC -> Map network drive

`\\rds.imperial.ac.uk\RDS\user\your_username_goes_here`



- if you don't have College-managed PC, you may need to use "**IC\username**" when loging in.

1b. Connect to RDS with File Explorer on Windows

Detailed description of connecting to RDS from Windows File Explorer:

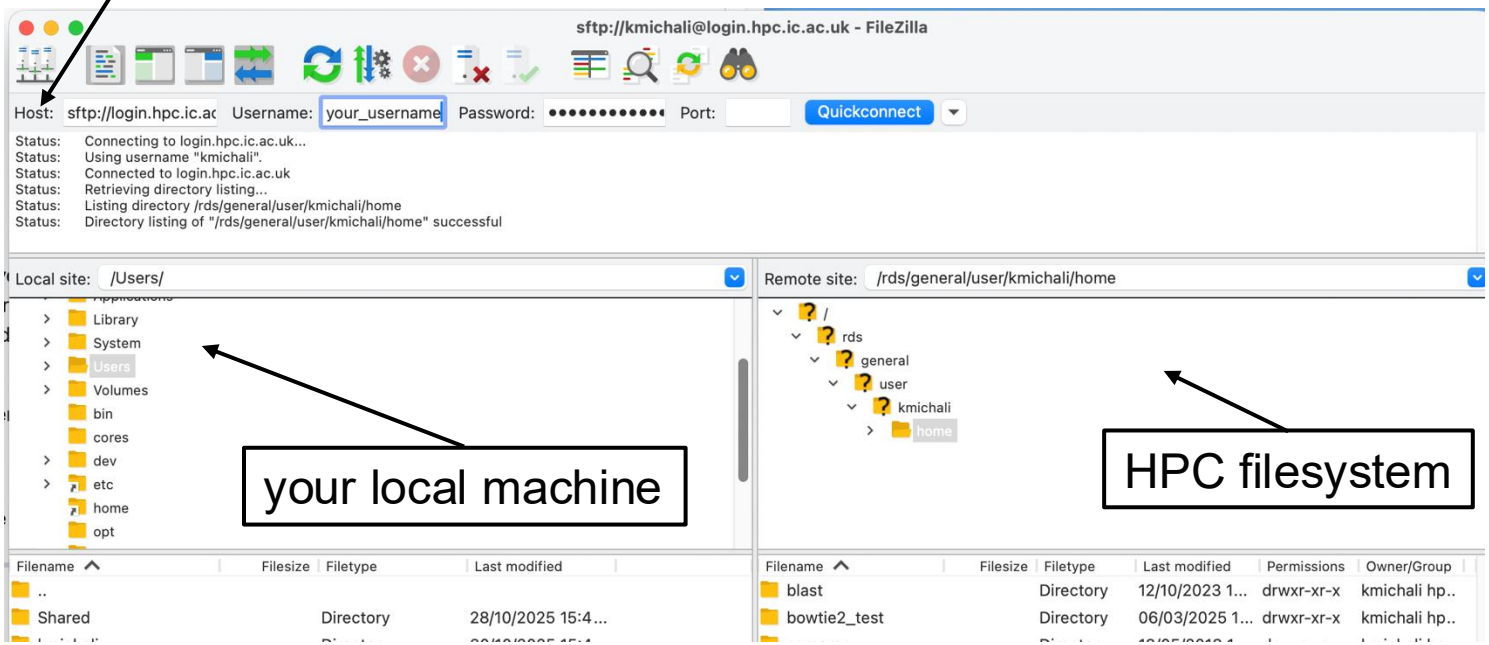
https://github.com/ImperialCollegeLondon/RCS_UserSupport_public/tree/main/RDS_FSmounting/MS_Windows

2. Copying files between your laptop to RDS with FileZilla

- if on laptop, use **Imperial-WPA wifi** or **VPN** outside the College
- **FileZilla** file transfer client: https://filezilla-project.org

  type **sftp://login.cx3.hpc.ic.ac.uk** into the host window,

  (add username, password, leave port empty, click Quickconnect)



- **WinSCP – good alternative for Windows** https://winscp.net/

3. Copying files on the command line

Secure copy (scp) command will copy files between remote systems

- **scp file.txt username@login.cx3.hpc.ic.ac.uk:~**

this copies file.txt from the current directory on your machine to your home directory on RDS

- **scp username@login.cx3.hpc.ic.ac.uk:~/file2.txt .**

this copies a file2.txt from your home directory on RDS to the current directory on your machine

- The scp command can be used with any directory path, for example:

scp /Users/katerina/Desktop/file3.txt username@login.hpc.ic.ac.uk:/rds/general/user/kmichali/home/projectx

Demystifying HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
  - working remotely and login
  - software management on the cluster
  - copying files to and from the cluster
- software execution on the cluster
- interactive resources

Your command

Assembling the command line is your responsibility, it requires some knowledge of the particular package. We can help with making sure the application runs correctly and advise you on managing your files. We can comment on individual applications to some degree (depending on having the expertise in the group).

Examples:

- **python** python_code.py –i input.txt –o output.txt
- **matlab** –batch matlab_script
- **R** CMD BATCH r_script.R
- **bash** shell_script.sh
- **g09** water.com > log
- **blastall** -i query -d fasta_db -o output
- mpiexec **nwchem** input.nw > log

Running software

Do not execute your program from the command line.

Why?

When you log into the cluster, you find yourself on one of four login nodes. If all users compute here, the nodes would be overwhelmed and the rest of the cluster would be idle. Instead, instruct the **cluster resource manager** to execute your job.

Cluster resource manager

The resource manager takes care of:

- keeps track of available hardware resources and how busy they are
- sorting your requests into job classes
- scheduling your requests
- starting jobs when compute resources become available
- monitoring jobs
- producing logs

X

Talking to the resource manager

- Write **a shell script (or job script)** instructing the resource manager what to do for you. Passing your finished script to the resource manager is called “submitting a job”.

- The resource manager needs to know your **command** and also what **resources** your job needs.

  - Every job script must specify **run time, memory (RAM) and number of cores** needed to run the job.

  - If the requested resources are exceeded during the job run, the job is terminated.

  - The bigger the resource, the longer the wait for a job to start.

Imperial College London

Download practice repo

```
git clone https://github.com/kmichali/hellohpc.git
```

A job script

```
#!/bin/bash
#PBS -l walltime=00:10:00
#PBS -l select=1:ncpus=1:mem=1gb

ml tools/prod
ml Python

python $PBS_O_WORKDIR/hellohpc.py
```

job params

load software

your command

Submit and monitor a job

kmichali — kmichali@login-b:~/cx3_phase2/hellohpc — ssh kmichali@login-b.cx3.hpc.ic.ac.uk — 91×41

```
[kmichali@login-b hellohpc]$ cat submit.pbs
#PBS -l walltime=00:10:00
#PBS -l select=1:ncpus=1:mem=1gb

ml tools/prod
ml Python

python $PBS_O_WORKDIR/hellohpc.py
[kmichali@login-b hellohpc]$
[kmichali@login-b hellohpc]$
[kmichali@login-b hellohpc]$ qsub submit.pbs
173299.pbs-7
[kmichali@login-b hellohpc]$
[kmichali@login-b hellohpc]$
[kmichali@login-b hellohpc]$ qstat -u $USER

pbs-7:
                                                            Req'd  Req'd   Elap
Job ID          Username Queue    Jobname    SessID NDS TSK Memory Time  S Time
--------------- -------- -------- ---------- ------ --- --- ------ ----- - -----
173299.pbs-7    kmichali v1_smal* submit.pbs 16302*   1   1    1gb 00:10 E 00:00
[kmichali@login-b hellohpc]$
```

Resource manager commands

submit	**qsub** your_script	system returns a jobid
monitor	**qstat** qstat job_id	short overview of your jobs one job overview
delete	**qdel** jobid	caution: it takes a bit of time to see the job disappear from qstat output

Job states

B Array job has at least one subjob running

E Job is exiting after having run

F Job is finished

H Job is held

M Job was moved to another server

Q Job is queued

R Job is running

S Job is suspended

T Job is being moved to new location

U Cycle-harvesting job is suspended due to keyboard activity

W Job is waiting for its submitter-assigned start time to be reached

X Subjob has completed execution or has been deleted

Job status using selfservice

This page will list all your active jobs (queuing and running).

https://selfservice.rcs.imperial.ac.uk/jobs/qstat

Every job produces two log files

```
-bash-4.2$ ll *1037925
-rw------- 1 kmichali hpc-kmichali   0 Feb  5  2020 submit.pbs.e1037925
-rw------- 1 kmichali hpc-kmichali 462 Feb  5  2020 submit.pbs.o1037925
-bash-4.2$
```

error messages

screen output + res. manager messages

Imperial College London

Troubleshooting your jobs

All standard output (screen and errors - STDOUT and STDERR) from your job plus other useful info is written into log files. **Read the log files** carefully even if all seems fine.

Default names:

STDOUT - e.g. `submit.pbs.o9795758`

STDERR - e.g. `submit.pbs.e9795758`

If asking for help, use the ICT ASK form at https://www.imperial.ac.uk/admin-services/ict/self-service/research-support/rcs/get-support/contact-us/

provide as much information as your can to help us troubleshoot:

- job id
- steps to reproduce the error
- your script
- the logs (expected and observed)

Example of a python script (using SciPy bundle)

```
#!/bin/bash
#PBS -l walltime=01:00:00
#PBS -l select=1:ncpus=1:mem=1gb

ml tools/prod
ml SciPy-bundle

source ~/venv/your_env/bin/activate
cd $PBS_O_WORKDIR
python myscript.py
```

Example of a python script (using miniforge)

```
#!/bin/bash
#PBS -l walltime=01:00:00
#PBS -l select=1:ncpus=1:mem=1gb

eval "$(~/miniforge3/bin/conda shell.bash hook)”
conda activate my_env

cd $PBS_O_WORKDIR
python myscript.py
```

Example of a python script using a GPU

```
#!/bin/bash
#PBS -l walltime=01:00:00
#PBS -l select=1:ncpus=4:mem=24gb:ngpus=1

ml PyTorch/2.1.2-foss-2023a-CUDA-12.1.1
ml scikit-learn/1.3.1-gfbf-2023a
ml matplotlib/3.7.2-gfbf-2023a

cd $PBS_O_WORKDIR
python train_evaluate_iris.py
```

Example of a R script

```
#!/bin/bash
#PBS -l walltime=01:00:00
#PBS -l select=1:ncpus=1:mem=1gb

ml tools/prod
ml R-bundle-ICL

cd $PBS_O_WORKDIR
R CMD BATCH myscript.R
```

Matlab script

```
#!/bin/bash
#PBS -l walltime=01:00:00
#PBS -l select=1:ncpus=1:mem=1gb

ml tools/prod
ml MATLAB

cd $PBS_O_WORKDIR
matlab -batch myscript #you may need to tweak this command,
look up "running MATLAB on command line"
```

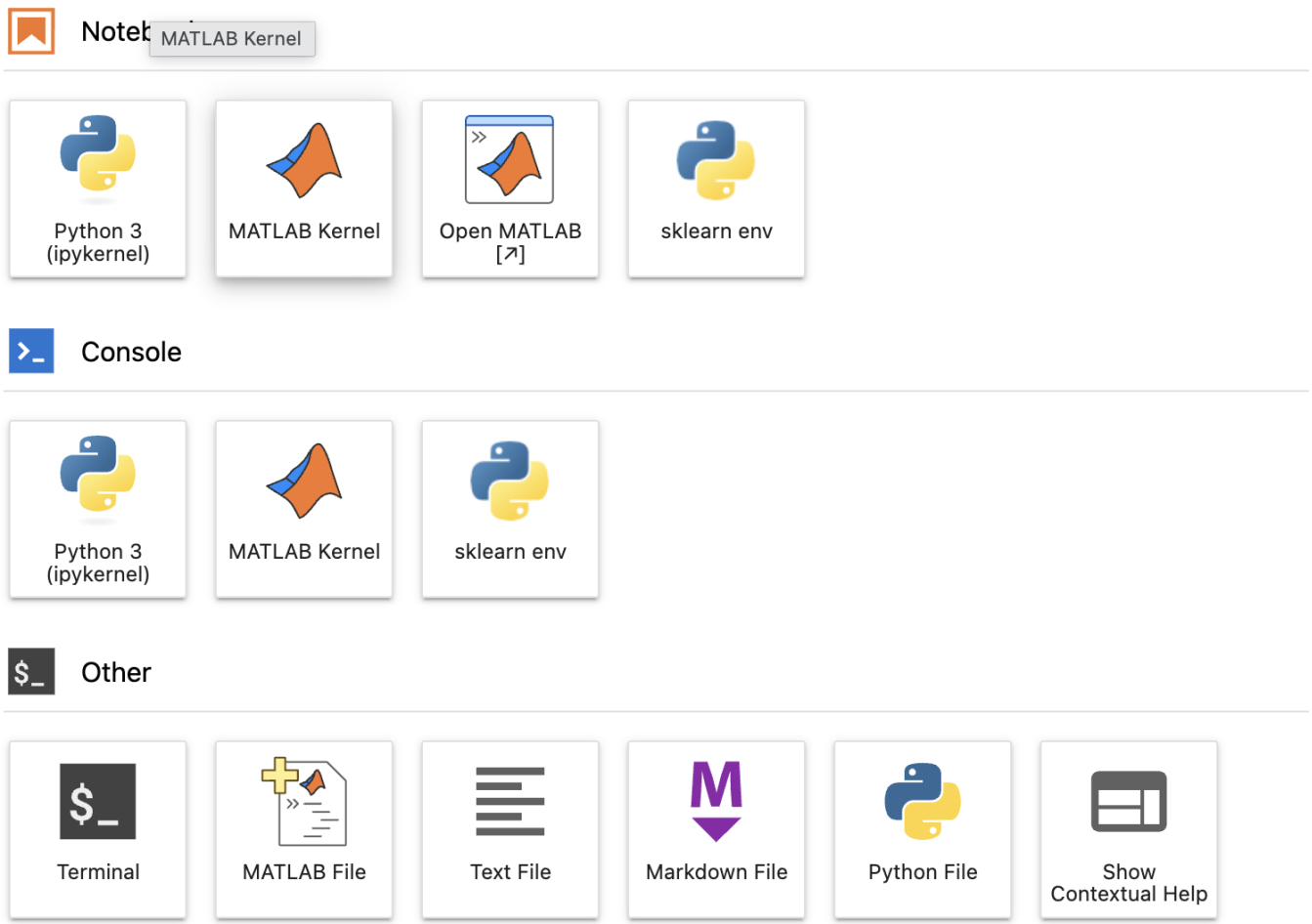
Demystifying HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
  - working remotely and login
  - software management on the cluster
  - copying files to and from the cluster
- software execution on the cluster
- interactive resources

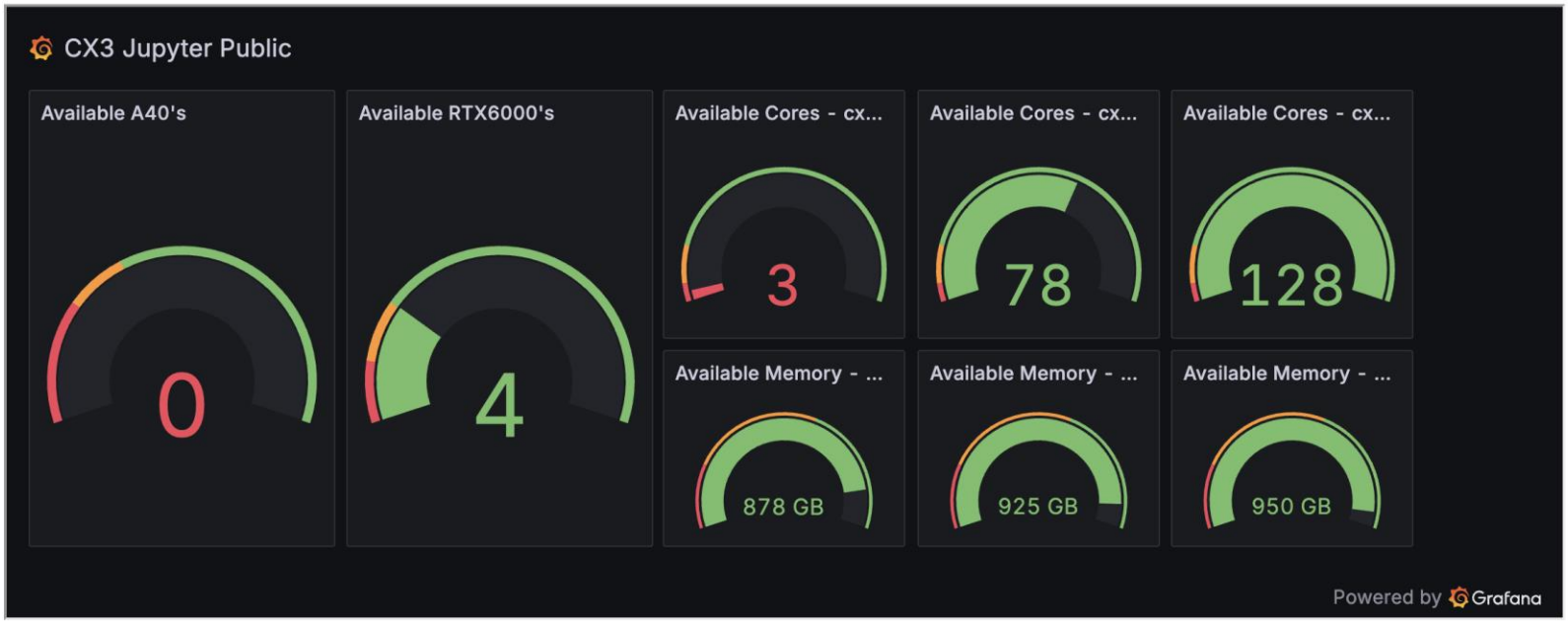
Interactive resources

- It is possible to use the HPC cluster via:
  - Jupyter Notebooks
  - RStudio

Jupyter Lab - https://jupyter.cx3.rcs.ic.ac.uk/



Jupyter lab job parameters



Select a job profile:

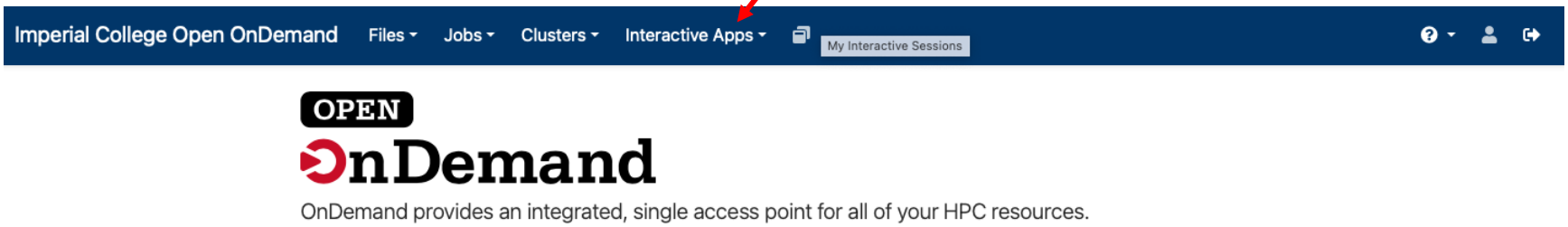
- ✓ 1 cores, 8GB, 2 hours
- 1 cores, 8GB, 4 hours
- 1 cores, 8GB, 8 hours
- 4 cores, 32GB, 8 hours
- 8 cores, 64GB, 8 hours
- 4 cores, 32GB, 8 hours, 1 NVIDIA A40 GPU
- 4 cores, 32GB, 8 hours, 1 NVIDIA RTX6000 GPU

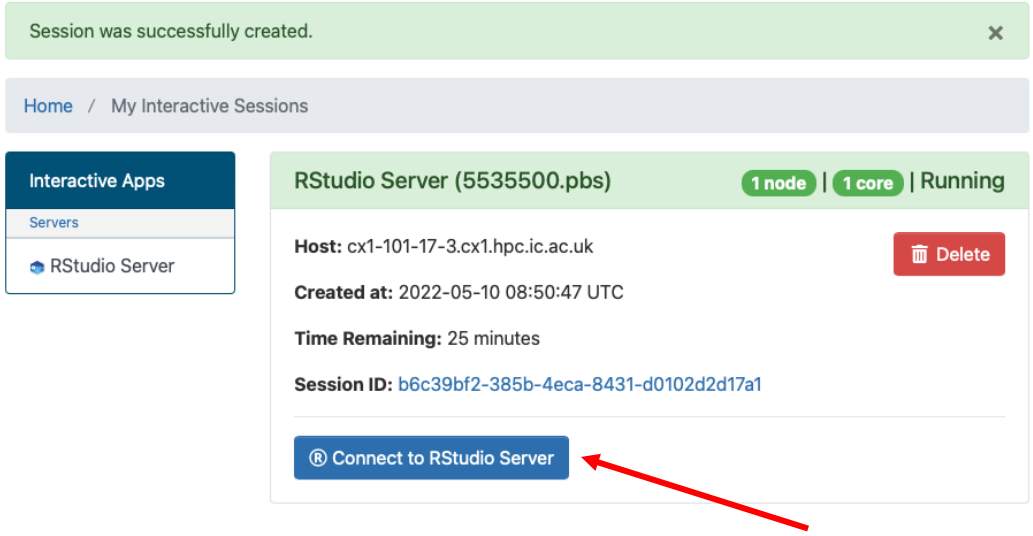
Imperial College London

RCS RESEARCH COMPUTING SERVICE
Gateway to Discovery

RStudio through Open OnDemand

https://openondemand.rcs.ic.ac.uk/





Thank you and happy computing!

Attribution-NonCommercial-ShareAlike

CC BY-NC-SA

This work is distributed under the terms of the Creative Commons Attribution-NonCommercial-ShareAlike license (CC BY-NC-SA) that lets others remix, adapt, and build upon this work non-commercially, as long as they credit the original source and license their new creations under the identical terms.

Imperial College London

RCS RESEARCH COMPUTING SERVICE
Gateway to Discovery

Introduction to HPC at Imperial

Katerina Michalickova – kmichali@imperial.ac.uk

Early Career Researcher Institute, Research Computing and Data Science

IMPERIAL
Early Career
Researcher Institute

Introduction to HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
  - working remotely and login
  - software management on the cluster
  - copying files to and from the cluster
- software execution on the cluster
- interactive resources
- short recap of fundamentals
- job parameters and job scripts
- managing input and output files in your script
- data parallelism
- parallel programs

Imperial College London

What can clusters do?

- the cluster can serve to offload code execution from your laptop/workstation
  - code that runs too long or needs too much memory or disk space
- clusters are particularly useful for executing parallel code
  - on one compute node
  - on multiple compute nodes at once

- Note on speed of execution:
- the compute nodes have similar architecture to your desktop - they are not much faster
- the main advantage of cluster computing lies in parallel code execution

HPC terminology

I will try to avoid or explain again during the class. In case I forget myself, here are few terms that need explanation:

- **Node** = compute node = computer in the cluster
- **Serial code** = runs on one core within the CPU
- **Parallel code** = program that runs on two or more cores
- **Job** = your program on the cluster
- **Submit job** = instruct the cluster to run your program

Key contacts

- RCS website - https://www.imperial.ac.uk/admin-services/ict/self-service/research-support/rcs/
- User support - https://www.imperial.ac.uk/admin-services/ict/self-service/research-support/rcs/get-support/contact-us/
- Technical documentation https://icl-rcs-user-guide.readthedocs.io/en/latest/
- Weekly drop-in clinics - https://icl-rcs-user-guide.readthedocs.io/en/latest/support/attend-a-clinic/
- Service status (under Zscaler)- https://selfservice.rcs.imperial.ac.uk/service-status
- Self-service (under Zscaler, interactive access, group management, job monitoring and extensions etc.) - selfservice.rcs.imperial.ac.uk

Log in using your terminal

Use Imperial-WPA or Zscaler.
In a terminal application, type the following:

```
ssh your_username@login.cx3.hpc.ic.ac.uk
```

- This course uses the cx3 system (aka cx3_phase2)
- The documentation:

https://icl-rcs-user-guide.readthedocs.io/en/latest/

Module system for centrally installed packages

- Lmod system sets required environment to use a software package

- List all modules available
  - ml av or ml ov (available or overview)
- Find a module for your software
  - ml av matlab or ml spider matlab
- Load a module
  - ml MATLAB
- Load a specific version
  - ml MATLAB/2024b
- List all loaded modules
  - ml
- Swap modules
  - ml switch MATLAB/2024b MATLAB/2023b
- Unload a module
  - ml unload matlab/R2018a
- Get rid of all loaded modules
  - ml purge

```
anaconda3/2.4.1
anaconda3/4.1.1
anaconda3/4.3.1(default)
anaconda3/personal
angsd/609
angsd/915
annovar/2013-03-18
annovar/2013-08-05
annovar/2015-06-17(default)
ansys/16.2
ansys/17.1
ansys/18.1
ansys/18.2
ansys/19.1-fluids
ansys/cfx/14.0(default)
ansys/cfx/15.0
ansys/cfx/16.1
ansys/fluent/14.0(default)
ansys/fluent/15.0
ansys/workbench/16.1
ant/1.6.1
ant/1.6.5
ant/1.8.1(default)
ants/2.2.0
ANTs/2015-02-23
aracne/2
armadillo/1.1.90
armadillo/1.2.0
armadillo/2.0.1
armadillo/3.2.4
armadillo/3.4.2
armadillo/6.200.2
armadillo/7.200.2(default)
arpack/96-patch(default)
aspect/1.4.0
aspect/1.5.0(default)
aster/11.3
atat/2.71(default)
ATK/2015.1
magma/2.18-5
magma/2011-07-05
magma/devel-modules(default)
make/3.82(default)
maker/2.10
maker/2.31.9
mamba/1.0.0
mantra/2012-11-19
maple/2016
mapsembler/2.2.4
mapsplice/2.2.0
maq/0.7.1(default)
marv/1.0.4
masurca/2.3.2
masurca/3.2.1
masurca/3.2.1_01202017
matlab/R2017a(default)
matlab/R2017b
matlab/R2018a
matplotlib/0.90.1(default)
mauve/2015-02-13
maxima/5.40.0(default)
mayavi/1.5(default)
mcarts/1.2.0
mcl/14.137
mcr/717(default)
mcscan/0.8
mcstas/2.1(default)
meep/0.10
meep/0.10.1
meep/0.20.3
meep/0.20.4
meep/1.0.3
meep/1.1.1(default)
meep/1.3
meep/1.4.3
meme/4.3.0
meme/4.3.0-nompi
meme/4.9.0
```

Python and R packages management

- You can manage R and Python either with Anaconda or Easybuild:

- **Anaconda package manager**
  - Simpler to use, possible not as performant, and prone to version resolution issues
  - You install all software yourself
- **Centrally installed software through Easybuild**
  - Always optimised, slightly more complicated setup
  - Most is installed for you, sone you do yourself (via pip or inside R)

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/applications

Your files on the cluster

All computers in the HPC resource are connected to one parallel filesystem – Research Data Store (RDS). Transfer your files via network drive mapping, command line scp, or multiple available packages (FileZilla, WinSCP etc).

You get access to:

your home directory ($HOME)	• personal working space • 1 TB allocation (up to 10 million files)
temporary storage ($EPHEMERAL)	• additional individual working space • 10TB • files deleted after 30 days
allocated project space ($RDS_PROJECT)	• your project allocations (if your supervisor has any)

check your usage with “quota -s”

documentation: https://icl-rcs-user-guide.readthedocs.io/en/latest/rds/

PBS cluster resource manager

The resource manager takes care of:

- keeps track of available hardware resources and how busy they are
- sorting your requests into job classes
- scheduling your requests
- starting jobs when compute resources become available
- monitoring jobs
- producing logs

X

Introduction to HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
  - working remotely and login
  - software management on the cluster
  - copying files to and from the cluster
- software execution on the cluster
- interactive resources
- short recap of fundamentals
- job parameters and job scripts
- managing input and output files in your script
- data parallelism
- parallel programs

Imperial College London

Download practice repo

```
git clone https://github.com/kmichali/hellohpc.git
```

A simple job script

```
[kmichali@login-c 20251030_intro_hpc]$ cat submit.pbs
#PBS -l walltime=00:10:00
#PBS -l select=1:ncpus=1:mem=1gb

ml tools/prod
ml Python

python $PBS_O_WORKDIR/hellohpc.py
[kmichali@login-c 20251030_intro_hpc]$
```

Requirements syntax

The system expects the following syntax at the top of your job script:

lower case L

job duration requirement

```
#PBS -l walltime=24:00:00
#PBS -l select=1:ncpus=1:mem=1gb
```

number of cores on one node

number of compute nodes

memory per node

Job duration requirement

- This specifies how long you expect the program to run.
- If you are not sure at all how long your job will run, sensible values are 24 hours, 48 hours or 72 hours. Otherwise choose time that you think your job will take plus some just to be sure.
- If possible, use checkpointing, i.e. save intermediate results that can be used to restart your program.

Memory requirement

- When you run a program, the program and data are read into the main memory (RAM). The data is processed in memory and eventually written out (usually into a file on the disk).

- Majority of programs that do not deal with lots of data need no more than 4GB of RAM. The data usually determines the bulk of the memory requirement. If your program reads large data files, ask for more memory.

- If in doubt, select 4GB to start with. If this is not enough, the resource manager will terminate your program and place a message (including how much RAM you tried to use) in a **log file**. In this case, increase the memory requirement and run again.

Number of parallel processes

- You program may be capable of parallel execution. If you are not sure, read the manual, ask your colleagues or check the HPC logs that specify how many cores your program tried to use. Failing that, visit the RCS clinic.

- On the HPC cluster, we generally see two types of parallelism:
  - shared memory (jobs that run on one node using threads)
  - distributed memory (jobs that run on multiple nodes using MPI)

Submit script for threaded parallel jobs

Threaded (OpenMP) jobs run on multiple cores on a single node - they require shared memory. In your script, increase ncpus count, memory (to accommodate more cores) and add ompthreads parameter (ompthreads is always equal or less than ncpus).

```
#PBS -l walltime=00:01:00
#PBS -l select=1:ncpus=4:mem=16gb:ompthreads=4
```

number of cores per node

memory per node

sets env. variable

Submit script for MPI jobs

MPI jobs don't require shared memory and run on multiple cores on multiple nodes. State the number of nodes using "select". The rest of the params (ncpus, mem and mpiprocs) state the requirement per one node; mpiprocs are always equal (or less than) ncpus.

number of cores on one node

```
#PBS -l walltime=24:00:00
#PBS -l select=2:ncpus=64:mem=128gb:mpiprocs=64
```

number of compute nodes

memory per node

sets enviroment

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/queues/mpi-jobs/

Submit script for GPU job

```
#PBS -l walltime=24:00:00
#PBS -l select=1:ncpus=4:mem=24gb:ngpus=1
```

number of cores on one node

memory per node

number of GPUs

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/queues/gpu-jobs/

Submit script for GPU job and specify GPU type

```
#PBS -l walltime=24:00:00
#PBS –l select=1:ncpus=4:mem=24gb:ngpus=1:gpu_type=RTX6000
```

GPU type

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/queues/gpu-jobs/

Possible job paramenters

Decide what your job needs and select your job params so your job fits the closest one of the following (wide) categories:

https://icl-rcs-user-guide.readthedocs.io/en/latest/hpc/queues/job-sizing-guidance/

Interactive jobs

- It is possible to have a direct access to bash shell on a compute cluster via an ”interactive” job.

- In terminal type (adapt your job requirement as you need):

qsub -I -l select=1:ncpus=2:mem=8gb -l walltime=02:00:00

- The job will queue and when it starts running you will have access to bash shell directly on a compute node.

kmichali — kmichali@cx3-2-13:~ — ssh kmichali@login-ai.cx3.hpc.ic.ac.uk — 110×45

```
[kmichali@login-ai hellohpc]$ qsub -I -l select=1:ncpus=2:mem=8gb -l walltime=02:00:00
qsub: waiting for job 178669.pbs-7 to start
qsub: job 178669.pbs-7 ready

[kmichali@cx3-2-13 ~]$ hostname
cx3-2-13.cx3.hpc.ic.ac.uk
[kmichali@cx3-2-13 ~]$
[kmichali@cx3-2-13 ~]$
```

Introduction to HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
  - working remotely and login
  - software management on the cluster
  - copying files to and from the cluster
- software execution on the cluster
- interactive resources
- job parameters and job scripts
- managing input and output files in your script
- data parallelism
- parallel programs

Current working directory for a job

When a job starts running, **the current working directory changes.** On cx3, it is your home directory.

This is somewhat counter-intuitive and can lead to file management errors.

For example, if you place your input file in the same directory as your job script and expect your program to read it in a current directory, the file will not be found.

The next few slides show different ways of managing this situation.

Hellohpc output

kmichali — kmichali@login-b:~/hellohpc — ssh kmichali@login-b.cx3.hpc.ic.ac.uk — 82×41

```
[kmichali@login-b hellohpc]$ cat submit.pbs.o173872
Hello from cx3-16-1-1.cx3.hpc.ic.ac.uk on Tue Feb  4 12:13:28 2025.
I'm job 173872.pbs-7 submitted by kmichali from
/rds/general/user/kmichali/home/hellohpc
and executed in
/rds/general/user/kmichali/home.

====================================
CPU Time used: 00:00:00
CPU Percent: 0%
Memory usage: 0b
Approx Power usage: 0.0
Walltime usage: 00:00:04

====================================
[kmichali@login-b hellohpc]$
```

The submit and execute directories are different

Job script with input and output files

This will not work because the default current directory for a job is not the same as the directory where the job was submitted (and where you expect to find your input file).

```
#PBS -l walltime=01:00:00
#PBS -l select=1:ncpus=1:mem=1gb

ml myprog

# input and output will not be found
# the current dir changes when the job is running

myprog input output
```

Environment variable pointing to your submit directory

$PBS_O_WORKDIR

- will exist when your job is running
- this is where you submitted your job
- usually a directory where your job script resides
- you can use this variable in your script

Theoretical job script using $PBS_O_WORKDIR

```
#PBS -l walltime=01:00:00
#PBS -l select=1:ncpus=1:mem=1gb

ml myprog


cd $PBS_O_WORKDIR
myprog input output
```

HelloHPC job script with an output file

```
#PBS -l walltime=00:10:00
#PBS -l select=1:ncpus=1:mem=1gb

ml tools/prod
ml Python

cd $PBS_O_WORKDIR
python hellohpc.py > mylog
```

Environment variable pointing to staging directory

$TMPDIR

- will exist when only when your job is running
- temporary space for your files - aka staging directory
- located on the node where the job is running
- same node = faster disk reading and writing

Staged jobs using $TMPDIR

If your job benefits from fast disk access, consider staging it, i.e. copy your data to staging directory ($TMPDIR), execute job there including writing output to staging dir. Move results back to submit directory at the end. Moving large datafiles at once using system commands is generally quite fast.

```
#PBS -l walltime=00:10:00
#PBS -l select=1:ncpus=1:mem=1gb
ml tools/prod
ml Python
#move python script into staging directory
cp $PBS_O_WORKDIR/hellohpc.py $TMPDIR
#run python script and write result in staging directory
python $TMPDIR/hellohpc.py > $TMPDIR/myresult.txt
#move result back to submit directory
mv $TMPDIR/myresult.txt $PBS_O_WORKDIR
```

Introduction to HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
  - working remotely and login
  - software management on the cluster
  - copying files to and from the cluster
- software execution on the cluster
- interactive resources
- job parameters and job scripts
- managing input and output files in your script
- data parallelism – array jobs
- parallel programs

Parallel computing

Data parallelism (array jobs)

- concurrent processing of data on an HPC cluster

Parallel programs

- parallelism is implemented on code level

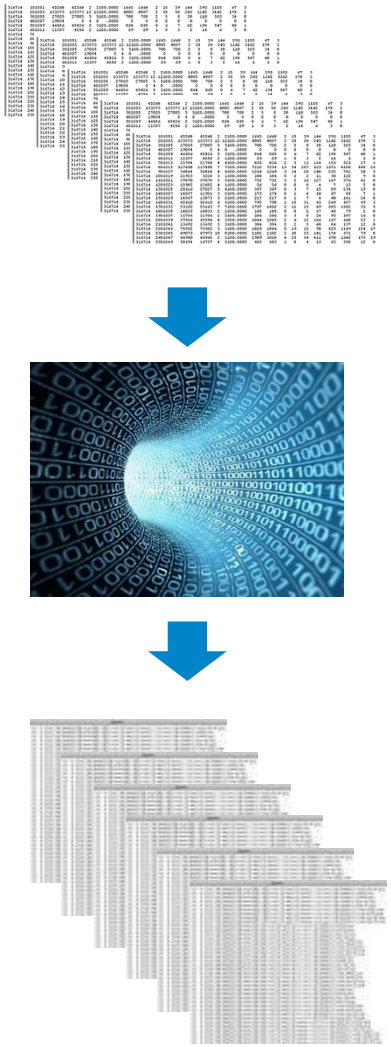
Data parallelism

Data parallelism examples:

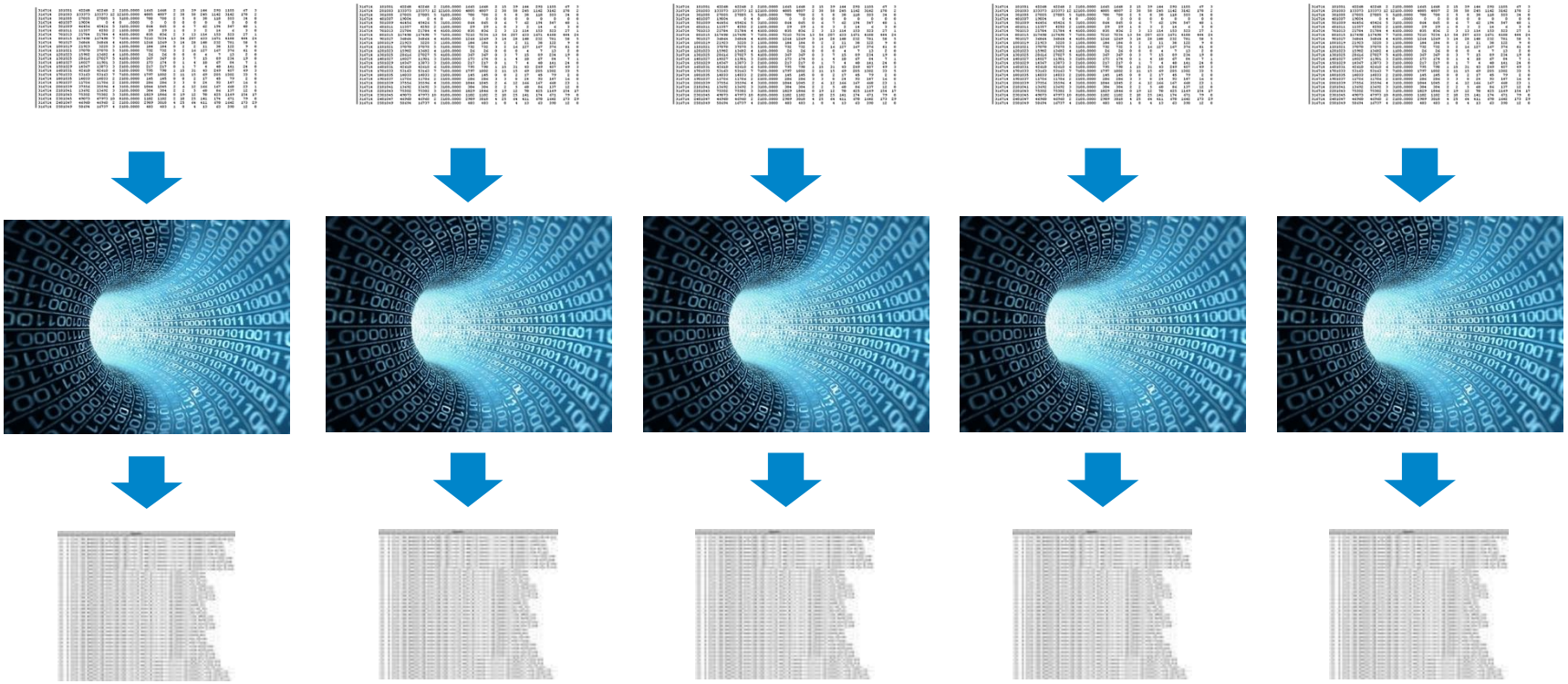
- processing multiple files
- processing large files that can be split
- simulation replicates
- parameter sweeps

Individual cases have to be independent:

- they do not exchange data
- input for any case doesn't depend on output of the other



Data parallelism – array jobs

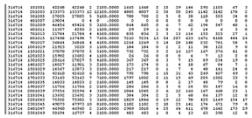


Advantages of data parallelism

- jobs run concurrently (in semi-parallel fashion)
- relatively easy to implement
- jobs run independently when a resource becomes available; no need to wait until a whole compute node is available (program-level parallel jobs would have to)

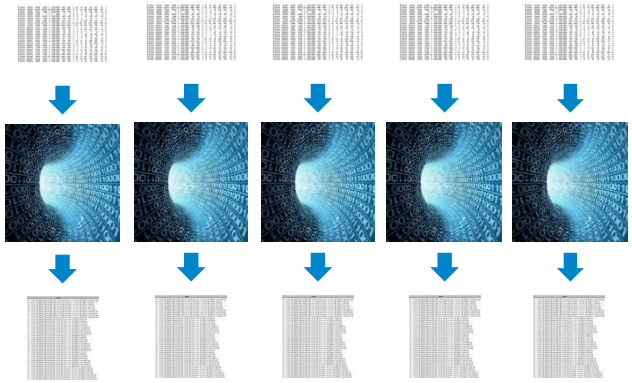
Theoretical single job (before parallelisation)

```
#PBS -l walltime=0:10:00
#PBS -l select=1:ncpus=1:mem=1gb


ml myprog

myprog input output
```



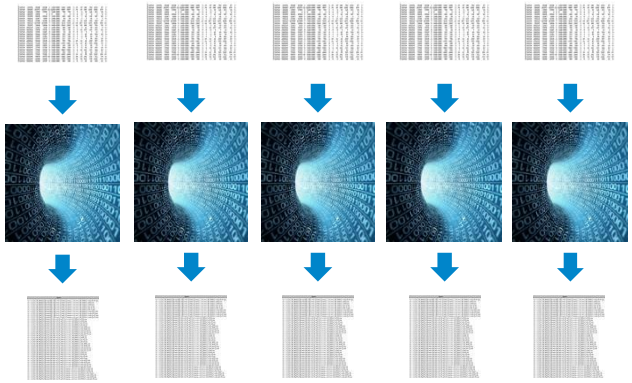
Parallelisation - array job script (theoretical)

```
#PBS -l walltime=0:10:00
#PBS -l select=1:ncpus=1:mem=1gb
#PBS -J 1-5

ml myprog

myprog  input? output?
```



Parallelisation - array job script (theoretical)

```
#PBS -l walltime=0:10:00
#PBS -l select=1:ncpus=1:mem=1gb
#PBS -J 1-5

ml myprog

myprog  input? output?
```



$PBS_ARRAY_INDEX takes values from 1 to 5 (or any other value specified by –J)

Array job scenario 1 – numbered files

Task: **run filter.py on infile;** BUT the file is too big and takes a long time to process.
Solution: Split infile to smaller chunks and process them concurrently.

```
-bash-4.2$ ll
total 32
-rw-r--r--. 1 kmichali hpc-kmichali 24471 Apr  4 13:56 infile
-bash-4.2$
-bash-4.2$
-bash-4.2$ split -n 3 -a 1 --numeric-suffixes=1 infile infile.
-bash-4.2$
-bash-4.2$
-bash-4.2$ ll
total 32
-rw-r--r--. 1 kmichali hpc-kmichali 24471 Apr  4 13:56 infile
-rw-r--r--. 1 kmichali hpc-kmichali  8157 Apr  4 14:13 infile.1
-rw-r--r--. 1 kmichali hpc-kmichali  8157 Apr  4 14:13 infile.2
-rw-r--r--. 1 kmichali hpc-kmichali  8157 Apr  4 14:13 infile.3
-bash-4.2$
```

Imperial College London

Hands-on – array jobs

Download and run array run case scenario 1:

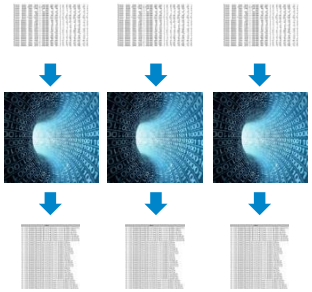
```
git clone https://github.com/kmichali/hellohpc.git
cd array/filter
qsub submit_filter_array.pbs
```

Array job script – numbered files (before parallelisation)

```
#PBS -l walltime=0:10:00
#PBS -l select=1:ncpus=1:mem=1gb
#PBS -N class

ml tools/prod
ml Python

cd $PBS_O_WORKDIR
python filter.py infile outfile
```

Array job script – numbered files

```
#PBS -l walltime=0:10:00
#PBS -l select=1:ncpus=1:mem=1gb
#PBS -J 1-3

ml tools/prod
ml Python

cd $PBS_O_WORKDIR
python filter.py infile.$PBS_ARRAY_INDEX outfile.$PBS_ARRAY_INDEX
```



$PBS_ARRAY_INDEX takes values from 1 to 3 (or any other value specified by –J)

Array job scenario 2 – file list

Task: **run stats.py on many molecules;** there are too many and it takes too long.
Solution: Process the molecules concurrently on the HPC cluster.

```
-bash-4.2$ ll
total 5
-rwxr-xr-x 1 kmichali hpc-kmichali   19 May 28 16:11 clean
-rw-r--r-- 1 kmichali hpc-kmichali 1158 May 28 16:11 cubane.pdb
-rw-r--r-- 1 kmichali hpc-kmichali  622 May 28 16:11 ethane.pdb
-rw-r--r-- 1 kmichali hpc-kmichali   69 May 28 16:11 list
-rw-r--r-- 1 kmichali hpc-kmichali  422 May 28 16:11 methane.pdb
-rw-r--r-- 1 kmichali hpc-kmichali 1828 May 28 16:11 octane.pdb
-rw-r--r-- 1 kmichali hpc-kmichali 1226 May 28 16:11 pentane.pdb
-rw-r--r-- 1 kmichali hpc-kmichali  825 May 28 16:11 propane.pdb
-rwxr-xr-x 1 kmichali hpc-kmichali  176 May 28 16:11 stats.py
-rw-r--r-- 1 kmichali hpc-kmichali  242 May 28 16:11 submit_list.pbs
-bash-4.2$
```

Imperial College London

Hands-on – array jobs

Download and run array run case scenario 2:

```
git clone https://github.com/kmichali/hellohpc.git
cd array/molecules
qsub submit_list.pbs
```

Array job script for one file (before parallelisation)

```
#PBS -l walltime=0:10:00
#PBS -l select=1:ncpus=1:mem=1gb

ml tools/prod
ml Python

cd $PBS_O_WORKDIR

INFILE=cubane.pdb
python stats.py $INFILE ${INFILE}.out
```

Leveraging $PBS_ARRAY_INDEX to choose input

```
katerina — ssh kmichali@login.hpc.ic.ac.uk — 89×36
-bash-4.2$ ls -l
total 5
-rwxr-xr-x 1 kmichali hpc-kmichali   19 Mar  6 15:24 clean
-rw-r--r-- 1 kmichali hpc-kmichali 1158 Mar  6 15:24 cubane.pdb
-rw-r--r-- 1 kmichali hpc-kmichali  622 Mar  6 15:24 ethane.pdb
-rw-r--r-- 1 kmichali hpc-kmichali  422 Mar  6 15:24 methane.pdb
-rw-r--r-- 1 kmichali hpc-kmichali 1828 Mar  6 15:24 octane.pdb
-rw-r--r-- 1 kmichali hpc-kmichali 1226 Mar  6 15:24 pentane.pdb
-rw-r--r-- 1 kmichali hpc-kmichali  825 Mar  6 15:24 propane.pdb
-rwxr-xr-x 1 kmichali hpc-kmichali  176 Mar  6 15:24 stats.py
-rw-r--r-- 1 kmichali hpc-kmichali  250 Mar  6 15:36 submit_list.pbs
-bash-4.2$ ls -1 *pdb
cubane.pdb
ethane.pdb
methane.pdb
octane.pdb
pentane.pdb
propane.pdb
-bash-4.2$ ls -1 *pdb | head -n 3
cubane.pdb
ethane.pdb
methane.pdb
-bash-4.2$ ls -1 *pdb | head -n 3 | tail -n 1
methane.pdb
-bash-4.2$ INFILE=$(ls -1 *pdb | head -n 3 | tail -n 1)
-bash-4.2$ echo $INFILE
methane.pdb
-bash-4.2$
```

The screen shows Linux commands that list the input files, pick the n-th file from the list, and store the file name in a variable.

These commands can be used in an array run job script to process a list of files.

The example picks the third file from the list and stores it in a variable called $INFILE.

Array job script – file list

```
#PBS -l walltime=0:10:00
#PBS -l select=1:ncpus=1:mem=1gb
#PBS -J 1-6

ml tools/prod
ml Python

cd $PBS_O_WORKDIR

INFILE=$(ls -1 *pdb | head -n $PBS_ARRAY_INDEX | tail -n 1)
python stats.py $INFILE ${INFILE}.out
```

Array job scenario 3 – for loops and matrices

- if you are running simulation replicates or parameter sweep, your program might contain a very time-consuming for loop.
- If the loop iterations are independent, there is a good chance that you can deploy it as an array run.
- The following Python example uses $PBS_ARRAY_INDEX value directly in the Python code.
- The code iterates through a matrix and uses individual elements for a calculation. In the original code, the calculations happen in serial manner.

Imperial College London

Hands-on – array jobs

Download and run array run case scenario 3:

```
git clone https://github.com/kmichali/hellohpc.git
cd array/matrix
qsub submit_matrix.pbs
```

Array job scenario 3 – for loops

Task: Iterate though a large matrix and use each element for a calculation using an array run.

Solution:

The following Python example uses $PBS_ARRAY_INDEX value directly in the Python code.

The code uses $PBS_ARRAY_INDEX to execute the correct iteration of the loop that processes just one element of the matrix.

All array runs combined process all elements of the matrix in parallel.

Python template for matrix iteration

```
input = np.loadtxt(”matrix.txt", dtype='f’,
delimiter=',’)
counter = 0

for i in range(0,input.shape[0]):
    for j in range(0, input.shape[1]):
        print('processing element', input[i,j])
        # your code for processing goes in here
```

Python template for matrix iteration with array run

```
input = np.loadtxt(”matrix.txt", dtype='f’,
delimiter=',’)
counter = 0
array_index = int(os.environ['PBS_ARRAY_INDEX'])

for i in range(0,input.shape[0]):
    for j in range(0, input.shape[1]):
        counter = counter+1
        if counter == array_index:
            print('processing element', input[i,j])
            # your code goes in here
```

Submit script for 3x3 matrix

```
#PBS -l walltime=00:10:00
#PBS -l select=1:ncpus=1:mem=1gb
#PBS -J 1-9


ml tools/prod
ml Python

cd $PBS_O_WORKDIR

python matrix_array.py
```

Caution - beware of too many files in one dir!

- HPC parallel file systems are sensitive to a large number of small files in one directory. The filesystem may slow down significantly in these circumstances and this will affect all users.

- If your array run is likely to produce > 10K files in one directory, you must adapt your script so it does not happen.

- In $PBS_O_WORKDIR, make directories for your logs. For example: ./out_logs and ./err_logs. In your script, redirect logs with:

```
#PBS -o ./out_logs -e ./err_logs
```

- Writing of your output files can be redirected to subdirectories. One possibility is to use $PBS_ARRAY_INDEX variable to name each subdirectory.

Managing log and output files for arrayruns

- This example uses PBS directives to redirect logs and makes a separate directory for results for each element of the array (do not forget to make directories for logs before you run a script):

```
#PBS -l walltime=00:10:00
#PBS -l select=1:ncpus=1:mem=1gb
#PBS -o ./out_logs -e ./err_logs
#PBS -J 1-6

ml tools/prod Python
cd $PBS_O_WORKDIR

#make a directory for your results
mkdir $PBS_ARRAY_INDEX

#run python script
python myscript.py input.$PBS_ARRAY_INDEX PBS_ARRAY_INDEX/output
```

Introduction to HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
- software execution on the cluster
- interactive resources
- job parameters and job scripts
- managing input and output files in your script
- data parallelism
- parallel programs on the cluster
  - OpenMP
  - MPI
  - Python and Julia

Parallel programs

So far, we have implemented parallelism using array runs on the cluster; the software that was used remained unchanged.

The next section is about parallelism that is achieved by writing parallel code.

Two types of parallel code:

- OpenMP or multiprocessing – requires shared memory
- Message Passing Interface – on multiple compute nodes at once

Parallel codes

- As it has become difficult and expensive to produce faster and faster processors, the focus has shifted in combining the power of multiple CPUs to work on a single task. We supports two main parallel programming paradigms.

- **OpenMP (C,C++,Fortran) and multiprocessing (Python)**

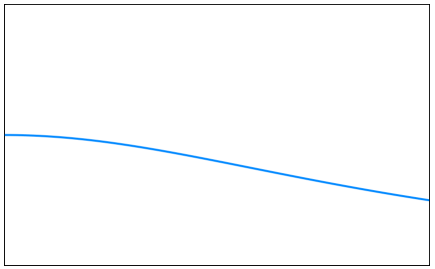
  OpenMP jobs require shared memory. The parallelization is achieved by using compiler directives around individual blocks of code, often loops or functions. Converting your serial program into OpenMP is “relatively” easier than attempting MPI programming.

- **Message Passing Interface (C, C++,Fortran, Python)**

  MPI jobs do not require shared memory and can be executed on separate cluster nodes. MPI standard consists of library routines that create parallel processes and pass necessary data (messages) between them. The messages are passed across the network if needed.

Real problem - approximate Pi with numerical integration

$$\frac{\pi}{4} = \int_0^1 \frac{dx}{1+x^2} \approx \frac{1}{N} \sum_{i=1}^{N} \frac{1}{1+\left(\frac{i-\frac{1}{2}}{N}\right)^2}$$

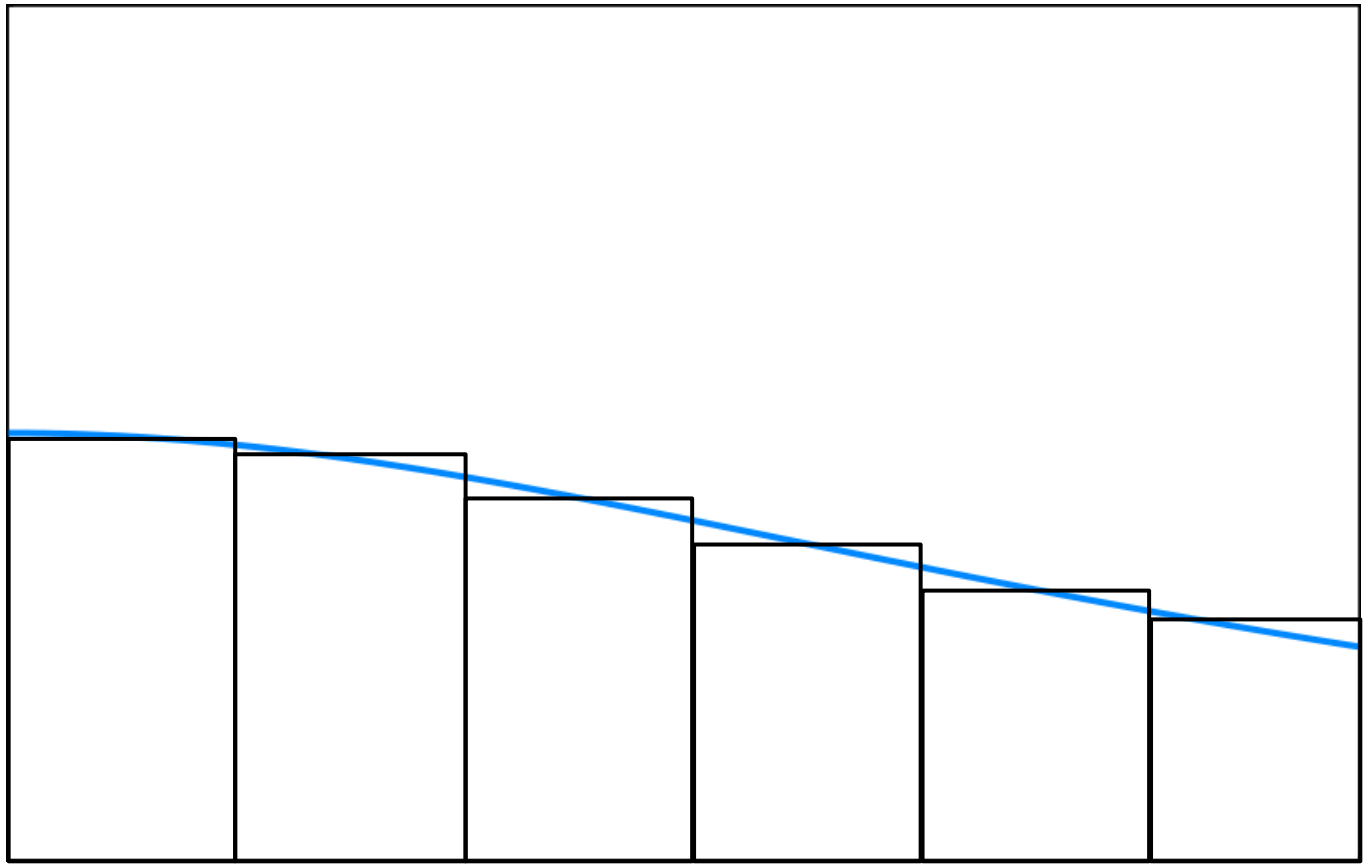


The answer becomes more accurate with increasing N

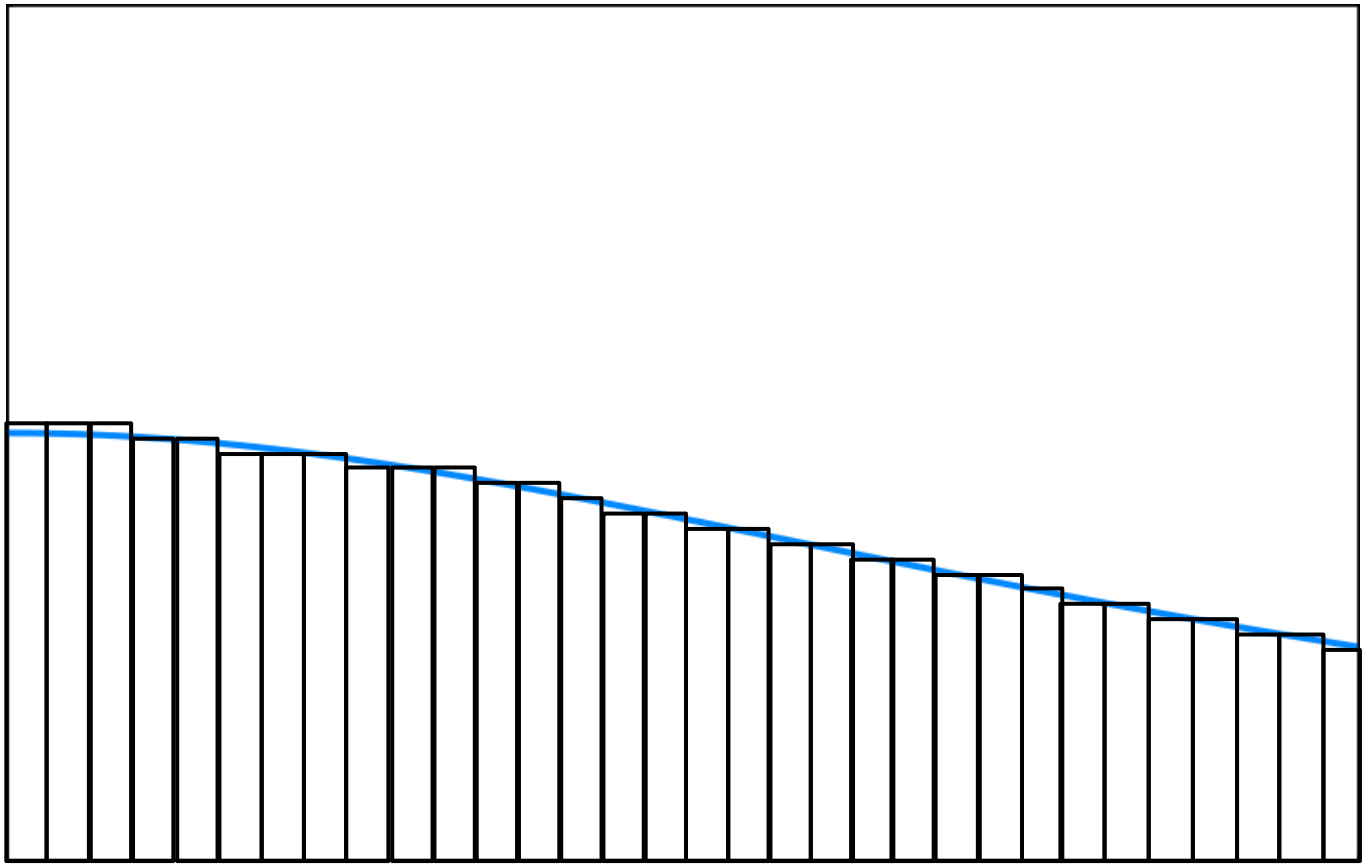
Iterations over N are independent, the calculation can be parallelised

We'll discuss pi code examples using serial, OpenMP, MPI and various python approaches.

Numerical integration



Numerical integration



Imperial College London

Serial Pi

$$\frac{\pi}{4} = \int_0^1 \frac{dx}{1+x^2} \approx \frac{1}{N}\sum_{i=1}^{N} \frac{1}{1+\left(\frac{i-\frac{1}{2}}{N}\right)^2}$$

```
program pi_serial

implicit none
integer(KIND=8)  :: n, i
double precision :: w, x, sum, pi, a, start, finish
double precision :: duration, timef
n=1E10
start=timef()

w=1.0/n
sum=0.0

do i=1,n
   x=w*(i-0.5)
   sum=sum+4.0/(1.0+x*x)
end do

pi=w*sum
finish=timef()
duration=finish-start
print*,n,"  ",pi,"  ",duration
end
```

Imperial College London

Hands-on – get codes, compile and run

```
#clone repository
git clone https://github.com/kmichali/hellohpc.git

# go to repository
cd parallel

# load compiler, parallel MPI library and compile
bash compile.sh

#run serial pi code
cd serial
qsub pi_serial.pbs
```

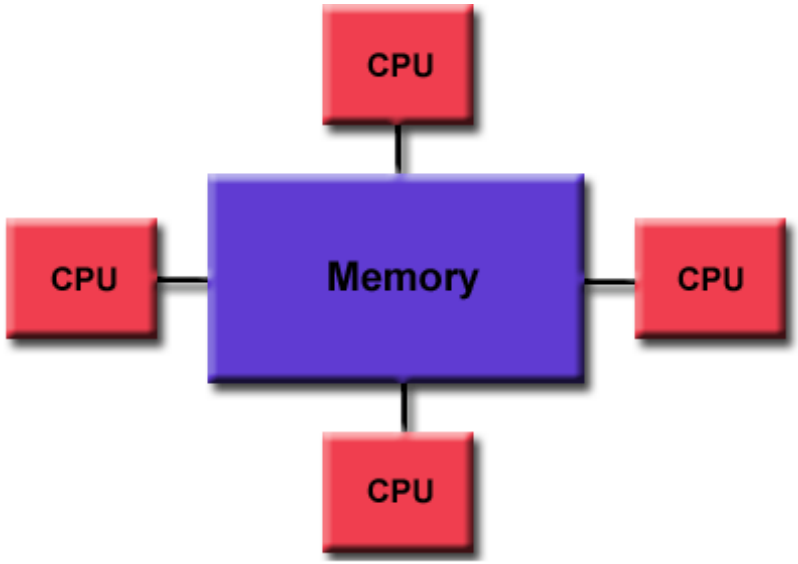
Introduction to HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
- software execution on the cluster
- interactive resources
- job parameters and job scripts
- managing input and output files in your script
- data parallelism
- parallel programs on the cluster
  - OpenMP
  - MPI
  - hybrid
  - Python and Julia

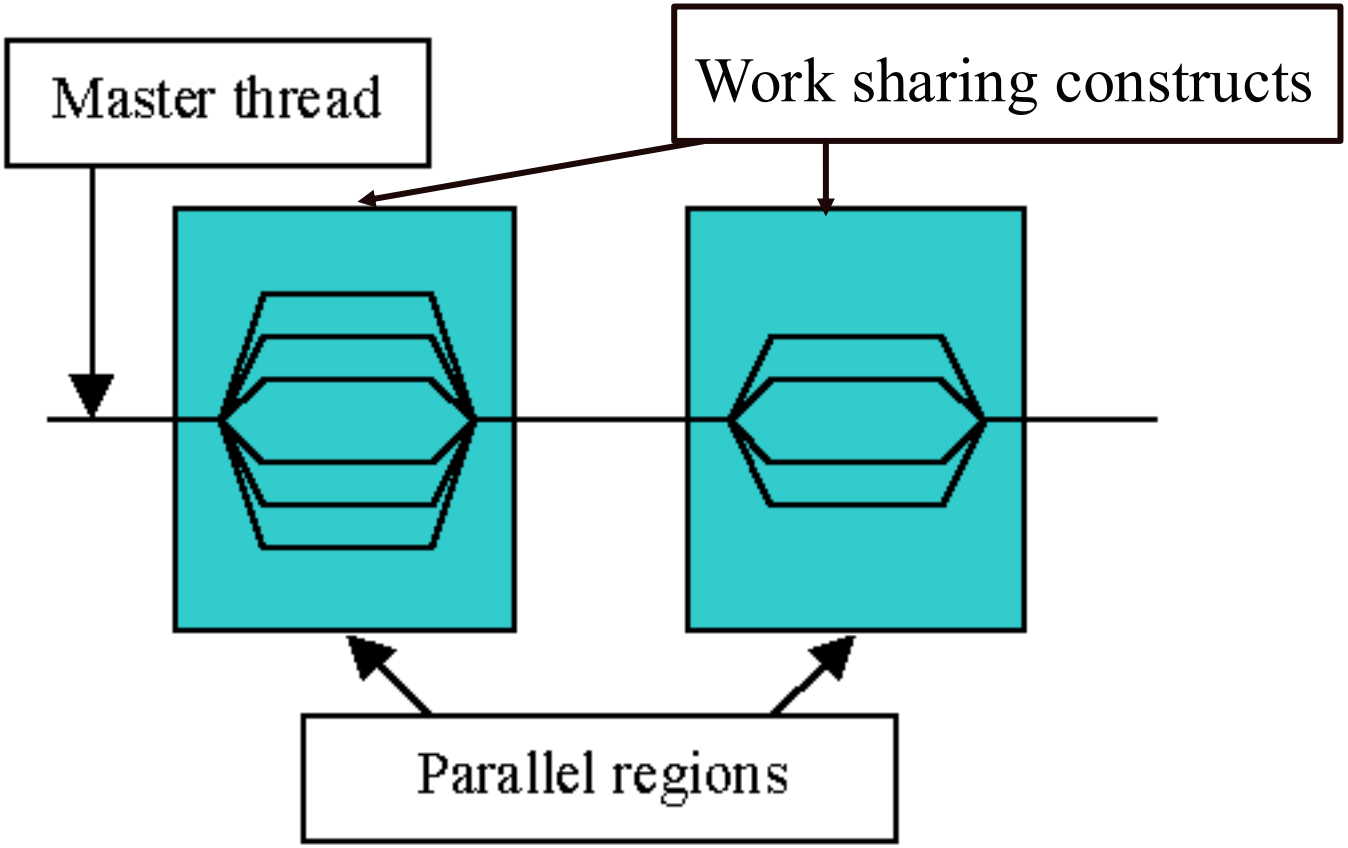
Parallel programming paradigms – OpenMP

OpenMP (open multiprocessing)

- set of extensions (compiler pragmas and API calls) to provide Fortran/C/C++ with the ability to run certain parts of the code in parallel, without explicitly managing (creating, destroying, assigning) threads
- requires shared memory



OpenMP



Hello world in C and OpenMP

```
#include <omp.h>
#include <stdio.h>
main (int argc, char *argv[])
{

  #pragma omp parallel
   {
       printf("Hello world!");
   }
}
```

OpenMP submit script

```
#PBS -l walltime=01:00:00
## Use 1 compute node with 8 cores and 4gb of memory
#PBS -l select=1:ncpus=8:mem=4gb:ompthreads=8

ml GCC

$PBS_O_WORKDIR/hello_openmp
```

Imperial College London

Hands-on – calculate Pi with OpenMP code

```
#run openMP pi code
cd ../openmp
qsub pi_openmp.pbs
```

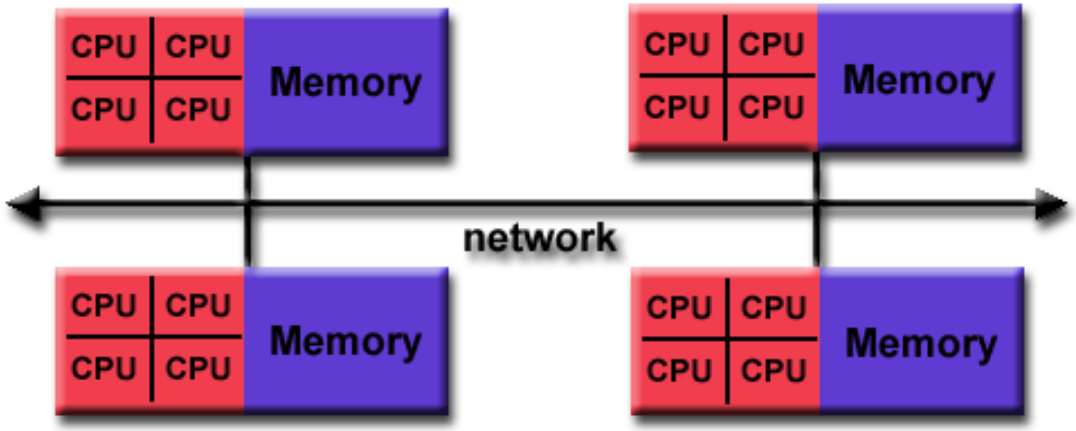
3.14159265358979323846 2643

Introduction to HPC at Imperial

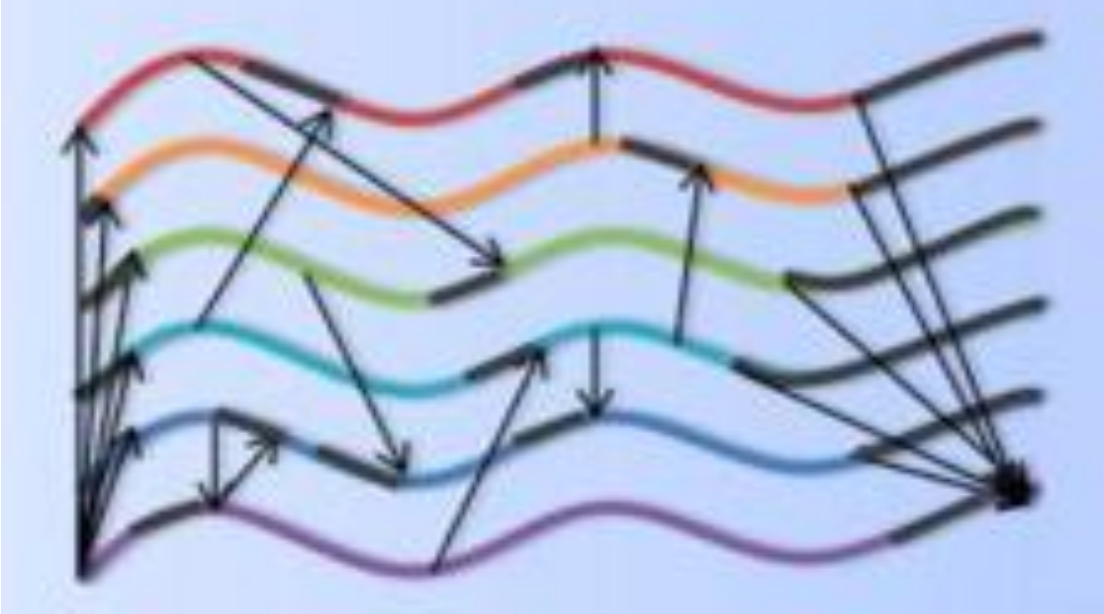
- introduction, key links and contacts
- preparing to use the HPC cluster
- software execution on the cluster
- interactive resources
- job parameters and job scripts
- managing input and output files in your script
- data parallelism
- parallel programs on the cluster
  - OpenMP
  - MPI
  - hybrid
  - Python and Julia

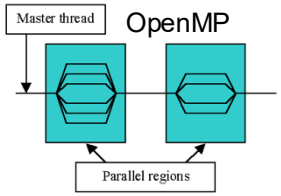
Message Passing Interface - MPI

- message passing parallel programming model
- one program is deployed on multiple nodes with distributed memory
- messages (data) are sent over the network
- MPI is a specification for a library with different implementations (Open MPI, Intel, MVAPICH)



MPI program





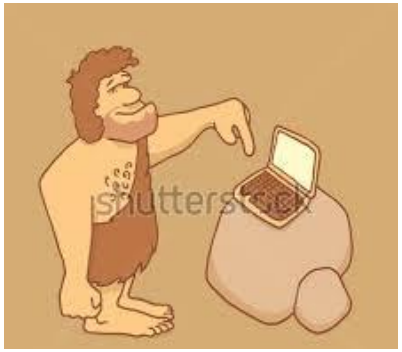
Hello world in C and MPI

```
int main(int argc,char **argv) {
  MPI_Comm comm;
  int nprocs,procid;

  MPI_Init(&argc,&argv);
  comm = MPI_COMM_WORLD;
  MPI_Comm_size(comm,&nprocs);
  MPI_Comm_rank(comm,&procid);

  int name_length = 100;
  char proc_name[name_length];
  MPI_Get_processor_name(proc_name,&name_length);


  printf("Hello from process %d out of %d running on %s\n",
       procid, nprocs, proc_name);


  MPI_Finalize();
}
```

MPI job submit script

```
#PBS -l walltime=01:00:00
## Use 2 nodes with 24 cores each and 4 gb of memory per node.
#PBS -l select=2:ncpus=24:mem=4gb:mpiprocs=24

ml GCC OpenMPI

mpiexec $PBS_O_WORKDIR/hello_mpi
```

Imperial College London



Hands-on 4 – calculate Pi with MPI code

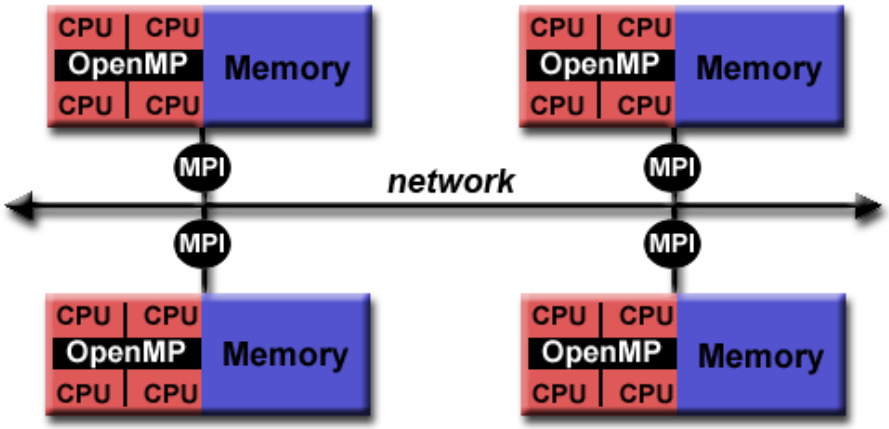
```
#run MPI pi code
cd ../mpi
qsub pi_mpi.pbs
```

Introduction to HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
- software execution on the cluster
- interactive resources
- job parameters and job scripts
- managing input and output files in your script
- data parallelism
- parallel programs on the cluster
  - OpenMP
  - MPI
  - hybrid
  - Python and Julia

Hybrid jobs

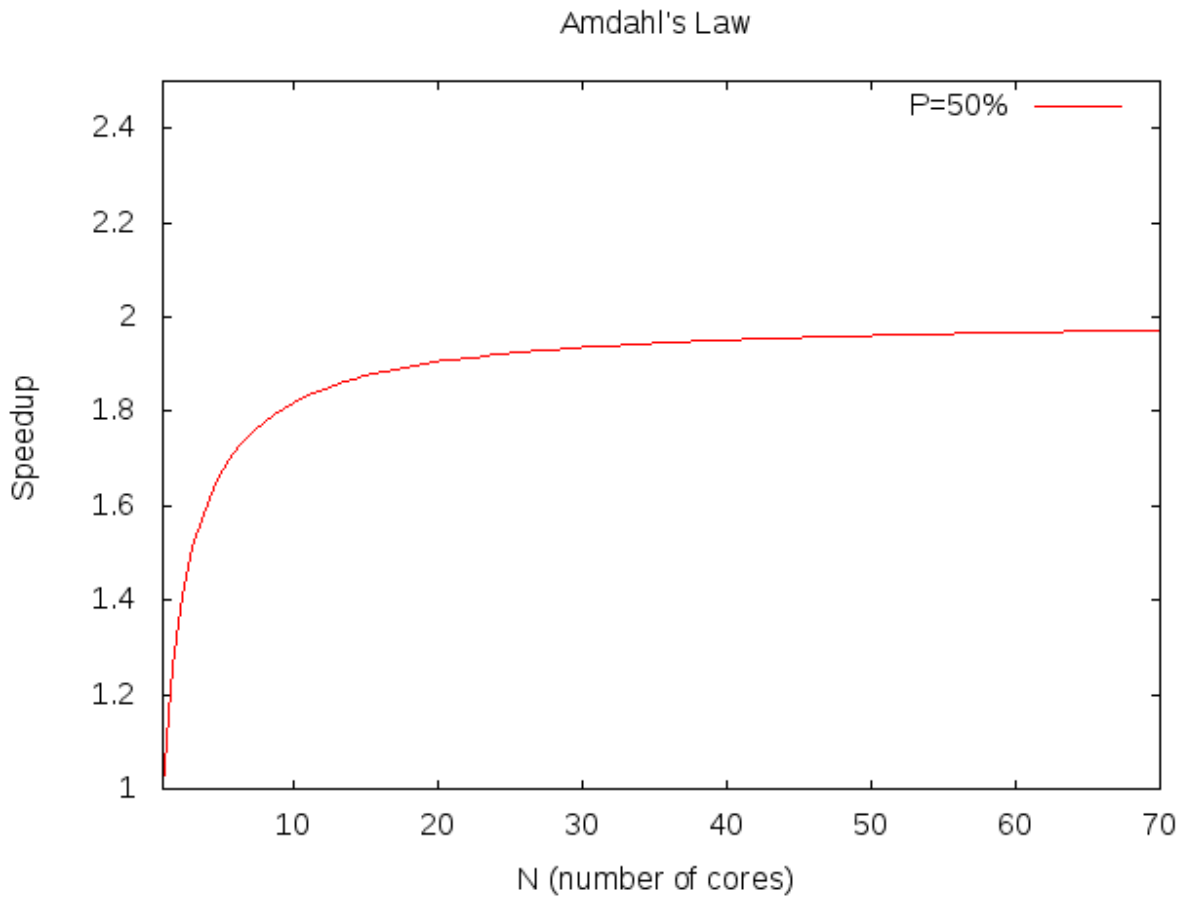
- hybrid code combines MPI with OpenMP
- MPI provides distributed memory parallelism
- OpenMP provides on-node shared memory parallelism
- the model reduces data movement within a node
- way to combine vectorisation and large scale parallel code



Hybrid submit script

```
#!/bin/sh
#PBS -l walltime=01:00:00
#PBS -l select=2:ncpus=24:mem=4gb:mpiprocs=1:ompthreads=24
## Use 2 nodes with 24 cores each and 4 gb of memory per node.
## Each node will host 1 mpirank and 24 threads.

ml GCC OpenMPI

mpiexec $PBS_O_WORKDIR/myprog
```

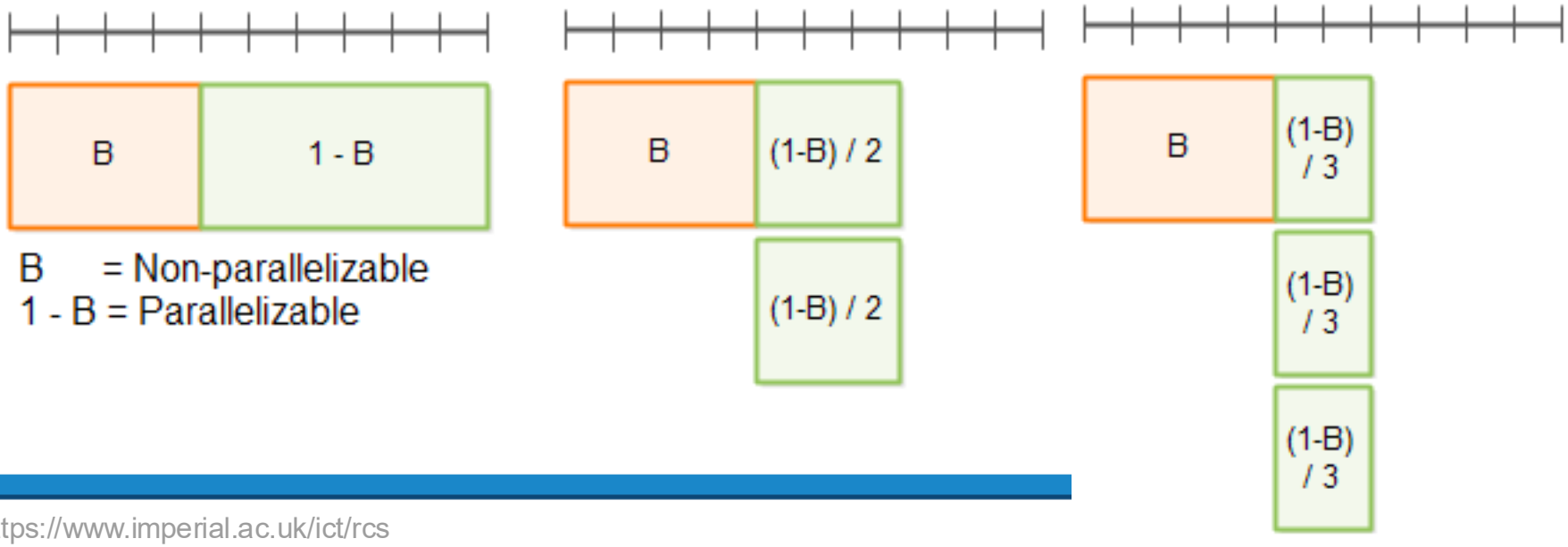
Amdahl's law


Amdahl's Law
P=50%
Speedup
2.4
2.2
2
1.8
1.6
1.4
1.2
1
10
20
30
40
50
60
70
N (number of cores)

Amdahl's law

Amdahl's law states that in parallelization, if 1-B is the proportion of a system or program that can be made parallel, and B is the proportion that remains serial, then the maximum speedup that can be achieved using N number of processors is 1/(B+((1-B)/N).

If N tends to infinity then the maximum speedup tends to 1/B.



Introduction to HPC at Imperial

- introduction, key links and contacts
- preparing to use the HPC cluster
- software execution on the cluster
- job parameters and job scripts
- managing input and output files in your script
- interactive resources
- data parallelism
- parallel programs on the cluster
  - OpenMP
  - MPI
  - Python and Julia

Parallel python?

- It is possible but not straightforward.
- The usual suspects – multiprocessing and MPI – were not the best way to go.

Pi code with python

- serial with python interpreter
- multiprocessing with Pool
- python MPI with mpi4py
- serial with pypy3 interpreter
- serial with just in time compiling (numba and jit)
- parallel with numba and jit

Imperial College London

Hands-on – get codes

```
#clone repository
git clone https://github.com/kmichali/hellohpc.git
cd parallel/python

#you will not be able to run the codes unless you build
python dependencies, installation help files are
included in the repository
```

Serial pi in Python – very slow

```
import time

n = int(10e9)
start_time = time.time()
w = 1.0/n
psum = 0.0

for i in range(1,n+1):
    x = w*(i - 0.5)
    psum = psum+4.0/(1.0 + x*x)

pi=w*psum
duration = time.time() - start_time
print(f"{n:,d}", "   ", pi, "   ", duration)
```

Multiprocessing with Pool – memory issues

```
import time
import multiprocessing as mp
import functools
import operator

def calc_sum(i):
    psum = 4.0/(1.0 + w*w*(i - 0.5)*(i - 0.5))
    return psum


n = int(10e9)
w = 1.0/n
start_time = time.time()

pool = mp.Pool(24)
result = pool.map(calc_sum, range(1,n+1))
total = functools.reduce(operator.add, result)
pi=w*total
print(f"{n:,d}", "   ", pi, "   ", time.time() - start_time)
```

MPI with mpi4py – mild improvement

```
import numpy as np
from mpi4py import MPI

n = int(10e9)

comm = MPI.COMM_WORLD
myrank = comm.Get_rank()
nproc = comm.Get_size()

if myrank == 0:
    start_time = time.time()

start = 1+myrank * int(n/nproc)
finish = (myrank+1) * int(n/nproc)

psum = 0.0
pi = 0.0
w = 1.0/n
```

```
for i in range(start, finish + 1):
    x = w*(i - 0.5)
    psum = psum+4.0/(1.0 + x*x)
```

```
psum = np.asarray(psum)
pi = np.asarray(pi)
comm.Reduce(psum, pi, op=MPI.SUM)
pi=w*pi

if myrank == 0:
    duration = time.time() - start_time
    print(n, "   ", pi, "   ", duration)
```

Numba and jit implementation - solution

```
import time
from numba import jit

@jit(nopython=True)
def calc_sum(n,w):
    psum = 0.0

    for i in range(1,n+1):
        psum += 4.0/(1.0 + w*w*(i - 0.5)*(i - 0.5))
    return psum


n = int(10e9)
start_time = time.time()
w = 1.0/n
sum = calc_sum(n,w)
pi=w*sum
print(f"{n:,d}", "   ", pi, "   ", time.time() - start_time
```

Parallel Numba – best solution

```
import time
from numba import jit, prange

@jit(nopython=True, parallel=True)
def calc_sum(n,w):
    psum = 0.0

    for i in prange(1,n+1):
        psum += 4.0/(1.0 + w*w*(i - 0.5)*(i - 0.5))
    return psum


n = int(10e9)
start_time = time.time()
w = 1.0/n
sum = calc_sum(n,w)
pi=w*sum
print(f"{n:,d}", "   ", pi, "   ", time.time() - start_time)
```

Julia – serial performance comparable to Fortran

```
function calc_pi(n)
    w = 1.0/n
    psum = 0.0

    for i in 1:n
        x = w * (i - 0.5)
        psum += 4.0 / (1.0 + x * x)
    end
    pi = w * psum
    return pi
end

using BenchmarkTools

n = 10_000_000_000
println(n)
println(calc_pi(n))
@btime calc_pi(n)
```

Thank you!

Attribution-NonCommercial-ShareAlike

CC BY-NC-SA

This work is distributed under the terms of the Creative Commons Attribution-NonCommercial-ShareAlike license (CC BY-NC-SA) that lets others remix, adapt, and build upon this work non-commercially, as long as they credit the original source and license their new creations under the identical terms.